

Heroes in Super-Training

Progettazione e validazione di un sistema ibrido basato sulla conoscenza per la classificazione di eroi multiverso.

Anno Accademico 2025/2026

Gruppo di lavoro: Michela Giulia Cericola, 778929, m.cericola@studenti.uniba.it
[Repository del progetto](#)

Indice

1	Introduzione	3
2	Sommario	3
3	Argomenti di Interesse	3
4	Dataset	3
4.1	Sommario	3
4.2	Strumenti Utilizzati	4
4.3	Decisioni di Progetto	4
4.4	Valutazione	4
5	Apprendimento Supervisionato	4
5.1	Sommario	4
5.2	Strumenti Utilizzati	5
5.3	Decisioni di Progetto	5
5.4	Valutazione	7
6	Ontologia	11
6.1	Sommario	11
6.2	Strumenti Utilizzati	11
6.3	Decisioni di Progetto	11
6.4	Valutazione	13
7	Sistema di Supporto alle Decisioni	16
7.1	Sommario	16
7.2	Strumenti Utilizzati	16
7.3	Decisioni di Progetto	17
7.4	Valutazione	18
8	Conclusioni e Sviluppi futuri	19
9	Bibliografia	21

1 Introduzione

Heroes in Super-Training è un progetto finalizzato a definire i ruoli degli eroi del multiverso DC e Marvel. La scelta di questo caso di studio è nata dal desiderio di mettermi alla prova con un dominio narrativo stimolante, scoprendo come la logica matematica potesse fare ordine in un immaginario così vasto.

2 Sommario

Il programma ha come obiettivo di sviluppare e testare comparativamente un'architettura ibrida in grado di coniugare l'apprendimento statistico induttivo con il rigore della logica simbolica e del ragionamento automatico.

Lo scenario d'analisi del progetto è focalizzato sul Multiverso dei supereroi, con particolare riferimento ai personaggi appartenenti ai due principali universi: DC Comics e Marvel.

Il sistema mira a superare i limiti dei modelli di Machine Learning, attraverso l'integrazione della Background Knowledge proveniente dal Web Semantico. Il sistema inoltre impiega la conoscenza strutturata dell'ontologia per alimentare un'interfaccia interattiva in cui l'utente può sottoporre il sistema a scenari operativi e sfide tattiche da superare.

3 Argomenti di Interesse

Gli argomenti di riferimento del corso su cui si articola il progetto sono:

- **Apprendimento Supervisionato (Machine Learning):** l'approccio data-driven è incentrato sull'addestramento di un albero di decisione per la classificazione statistica dei ruoli dei personaggi in base alle feature numeriche.
- **Rappresentazione della Conoscenza (Ontologia OWL) e Ragionamento Deduttivo:** l'ontologia modella formalmente la conoscenza di fondo del multiverso, mentre il ragionatore deduttivo (HermiT) applica le regole logiche per convalidare e correggere le risposte del Machine Learning.
- **Sistema di Supporto alle Decisioni (DSS):** modulo finale di interazione in cui il dominio viene convertito in una rappresentazione proposizionale. Il sistema permette all'utente di configurare scenari operativi e sfide tattiche, valutando e ordinando i personaggi tramite una funzione euristica per trovare la soluzione ottimale al problema proposto.

4 Dataset

4.1 Sommario

Il primo e fondamentale passo per la realizzazione del progetto Heroes in Super-Training è stato lo sviluppo del dataset da zero. Una ricerca preliminare sul web e sui repository pubblici (es. Kaggle) ha evidenziato l'assenza di basi di dati strutturate con attributi idonei a supportare l'idea di base del progetto.

La maggior parte dei dataset disponibili, infatti, presentava metadati prettamente biografici o narrativi destrutturati, privi di una conformazione matematica adatta a un compito di classificazione statistica supervisionata. Per addestrare un modello predittivo capace di associare

un supereroe al suo ruolo tattico, era invece necessaria una matrice di dati precisa, in cui ogni individuo fosse descritto da caratteristiche confrontabili.

4.2 Strumenti Utilizzati

Il dataset è stato realizzato tramite sistema di gestione di database relazionali open-source ([MySQL](#)). Consecutivamente sono stati esportati i dati nel file csv presente nel progetto.

4.3 Decisioni di Progetto

La struttura del dataset è stata definita scegliendo attributi che consentissero di rappresentare differenti aspetti dei personaggi e, allo stesso tempo, di costruire regole semantiche significative all'interno dell'ontologia.

Gli attributi utilizzati sono:

Id - identificativo univoco del super eroe;

name - nome dell'eroe, valore variabile con soglia 25 caratteri;

universe - universo dell'eroe, scelta tra due mondi (DC e MC). Utile nella terza fase del progetto in cui vediamo confrontarsi i due mondi;

strenght - forza dell'eroe, intero vincolato da 1 a 10;

intelligence - intelligenza dell'eroe, intero vincolato da 1 a 10;

speed - velocità dell'eroe, intero vincolato da 1 a 10;

role - ruolo dell'eroe, valore tra Leader, Powerhouse (inteso come potenza), Specialist. Il ruolo rappresenta la variabile target utilizzata nella classificazione supervisionata e costituisce inoltre un riferimento per la valutazione della componente ibrida;

power source - provenienza del potere dell'eroe, scelta tra soprannaturale e arma tecnologia;

popularity - popolarità dell'eroe, intero vincolato da 1 a 10;

La conoscenza personale di tale argomento e la ricerca sul web di alcuni personaggi che non mi erano noti, sono stati di supporto per la popolazione della tabella e dei valori in essa stessa.

4.4 Valutazione

Il dataset costituisce la base comune delle due componenti del sistema. Da un lato, le caratteristiche numeriche vengono utilizzate per l'addestramento dei modelli di Machine Learning; dall'altro, le stesse informazioni vengono trasformate in asserzioni della Knowledge Base e utilizzate dal reasoner per ottenere nuove informazioni.

5 Apprendimento Supervisionato

5.1 Sommario

La prima componente del sistema è dedicata all'apprendimento supervisionato e ha l'obiettivo di prevedere il ruolo tattico dei personaggi a partire dalle caratteristiche presenti nel dataset.

I ruoli considerati sono *Leader*, *Powerhouse* e *Specialist*. La variabile *role* costituisce quindi la variabile target del problema di classificazione, mentre le caratteristiche descrittive dei personaggi vengono utilizzate come variabili di input.

5.2 Strumenti Utilizzati

Il linguaggio a cui fa riferimento la creazione di questo progetto è Python. Questa scelta è legata alla sua versatilità e alla disponibilità di un ricco ecosistema di librerie. Infatti, per questo modulo sono state utilizzate:

- **pandas (pd)**: gestisce il dataset caricando il file CSV e manipolando i dati tramite strutture tabellari;
- **numpy (np)**: esegue calcoli matematici rapidi su vettori, come il calcolo dell'accuratezza media;
- **scikit-learn (sklearn)** utilizza:
 - **DecisionTreeClassifier**: classifica i ruoli mediante albero di decisione;
 - **KNeighborsClassifier**: utilizzato come secondo modello di classificazione, basato sulla similarità tra i campioni;
 - **StandardScaler**: utilizzato nella pipeline del k-NN per la standardizzazione delle caratteristiche numeriche;
 - **Pipeline**: combina le operazioni di preprocessing e classificazione nel modello k-NN;
 - **export_text**: estrae in forma testuale le regole apprese dall'albero di decisione;
 - **accuracy_score**: calcola l'accuratezza delle predizioni;
 - **cross_val_score**: utilizzato per la valutazione mediante cross-validation;
 - **KFold**: suddivide il dataset in cinque fold e realizzare la procedura di validazione incrociata.
- **matplotlib.pyplot (plt)**: genera e mostra i grafici visivi per analizzare l'andamento e i risultati delle simulazioni.

5.3 Decisioni di Progetto

In questa fase del progetto, si procede inizialmente con l'importazione del dataset tramite la lettura del file CSV precedentemente generato. Successivamente, viene eseguita un'operazione di preprocessing dei dati, applicando una codifica per convertire l'attributo qualitativo **power source** in valore numerico, rendendolo così elaborabile dall'algoritmo di classificazione.

Gli attributi selezionati sono: *strength*, *intelligence*, *speed*, *power_source*.

Queste caratteristiche giocano un ruolo fondamentale nel progetto, poiché permettono di definire metriche quantitative e qualitative precise, necessarie all'algoritmo per discriminare e comprendere al meglio il ruolo di ciascun eroe (*Leader*, *Powerhouse*, *Specialist*).

Per la predizione del ruolo tattico degli eroi ci serviamo di due differenti modelli di classificazione:

- **Decision Tree (Albero di Decisione)**: L'apprendimento del modello esegue una ricerca euristica per identificare i punti di scissione ottimali. Questo processo viene applicato in

modo ricorsivo top-down (dall'alto verso il basso): partendo dal dataset completo dei 130 eroi nel nodo radice, l'algoritmo seleziona l'attributo più discriminante in base al criterio di Gini per effettuare i vari tagli, arrestando la crescita alla profondità massima stabilita (`max_depth = 3`).

- **k-Nearest Neighbors (k-NN):** A differenza dell'albero, il k-NN è un algoritmo basato su istanze (*lazy learning*) che non costruisce un modello esplicito in fase di addestramento, ma memorizza i dati di training. Prima del calcolo delle distanze, le feature degli eroi (*strength*, *intelligence*, *speed*, *power_source_encoded*) vengono scalate mediante `StandardScaler` (con media pari a 0 e varianza 1). Questo passaggio è fondamentale affinché gli attributi con valori numerici più elevati non dominino il calcolo della distanza euclidea. Quando il sistema deve predire il ruolo di un eroe, calcola la distanza metrica nello spazio n -dimensionale tra quest'ultimo e tutti i 130 eroi già noti. L'algoritmo individua i k eroi numericamente più vicini (nel nostro caso $k = 5$, iperparametro selezionato tramite Cross-Validation) e assegna all'eroe da classificare il ruolo maggioritario tra quelli dei suoi 5 vicini più prossimi.

Perché la scelta del Decision Tree?

Il *Decision Tree Classifier* è stato eletto come algoritmo cardine per questo modulo di apprendimento supervisionato. Nella moderna Ingegneria dei Dati, l'adozione di modelli altamente complessi come Random Forest, Reti Neurali o architetture basate su istanze come il *k-Nearest Neighbors* (k-NN) garantisce buone prestazioni statistiche; tuttavia, ciascuno di essi presenta limiti strutturali rispetto agli obiettivi del progetto:

- **Trasparenza (White-Box) vs. Modelli Black-Box:** A differenza dei modelli complessi o del k-NN (che memorizza semplicemente le istanze nello spazio n -dimensionale), l'Albero di Decisione mappa in modo esplicito lo spazio delle *feature* in regole logiche booleane d'immediata comprensione (*IF - THEN*). Questa trasparenza è fondamentale per l'interoperabilità con il blocco del Web Semantico, poiché consente di estrarre le regole apprese e confrontarle direttamente con la logica descrittiva dell'Ontologia OWL.
- **Gestione della Scala e Indipendenza dal Preprocessing:** Mentre il k-NN richiede obbligatoriamente una fase di normalizzazione delle variabili (`StandardScaler`) per evitare che attributi su scale numeriche differenti distorcano il calcolo delle distanze euclidee, l'Albero di Decisione basato sull'indice di Gini (`criterion='gini'`) analizza le feature in modo isolato attraverso singoli punti di taglio (*threshold*). Ciò lo rende intrinsecamente immune alle differenti scale dei dati fisici (*strength*, *intelligence*, *speed*) e dei metadati qualitativi codificati (*power_source*).

```
decision_tree = DecisionTreeClassifier(  
    max_depth=3,  
    criterion='gini',  
    random_state=10  
)
```

Figura 1: Configurazione degli iperparametri del DecisionTreeClassifier.

L'algoritmo restringe la complessità strutturale imponendo una profondità massima di tre livelli (`max_depth=3`) per prevenire il fenomeno dell'*overfitting*. L'indice di Gini ottimizza i tempi di calcolo misurando l'impurità dei nodi e la probabilità di errata classificazione, adattandosi perfettamente alla natura ibrida del dataset.

Per valutare in modo rigoroso l'affidabilità di entrambi i modelli e giustificare empiricamente la superiorità del Decision Tree, si è optato per la tecnica di ricampionamento **5-Fold Cross Validation** ($k = 5$). Il dataset viene suddiviso in 5 parti uguali: ad ogni iterazione, 4 fold vengono impiegati per l'addestramento e il fold rimanente per la validazione.

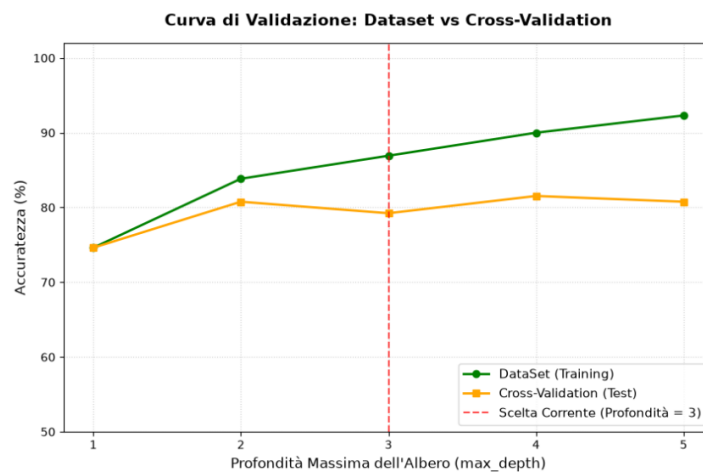
Infine, l'analisi quantitativa e il confronto delle performance al variare degli iperparametri (la profondità per l'albero e il numero di vicini k per il k-NN) sono supportati da uno studio grafico mediante la libreria `matplotlib`.

5.4 Valutazione

Valutazione del Decision Tree

In fase di valutazione rispetto questa parte del progetto si evince: la scelta più ottimale per l'ampiezza dell'albero di decisione è 3.

Lo possiamo notare dalla visualizzazione del grafico sottostante:



Il grafico mostra l'andamento dell'accuratezza del modello al variare della profondità massima dell'albero (`max_depth`), confrontando le prestazioni sul set di addestramento (*Training*, linea verde) con quelle ottenute tramite la validazione incrociata (*Cross-Validation*, linea arancione).

L'analisi del diagramma evidenzia tre comportamenti fondamentali:

- **Underfitting per $\text{max_depth} \leq 2$:** con profondità ridotta, l'albero non ha la capacità strutturale di evidenziare la complessità dei dati. Le regole estratte sono troppo generiche per discriminare correttamente i tre ruoli tattici.
- **Overfitting per $\text{max_depth} \geq 4$:** aumentando la profondità, si nota una netta divergenza tra le due linee. La curva di *Training* continua a salire quasi linearmente, mentre la curva di *Cross-Validation* si stabilizza e presenta un accenno negativo a profondità 5. Vi è una perdita di generalizzazione.
- **Punto ottimale e trade-off ($\text{max_depth} = 3$):** identificato dalla linea tratteggiata rossa, profondità 3 rappresenta il perfetto punto di equilibrio del sistema. In questa configurazio-

ne, l'albero raggiunge un'ottima accuratezza di validazione (circa 85%) mantenendo uno scarto ridotto rispetto alla curva di addestramento.

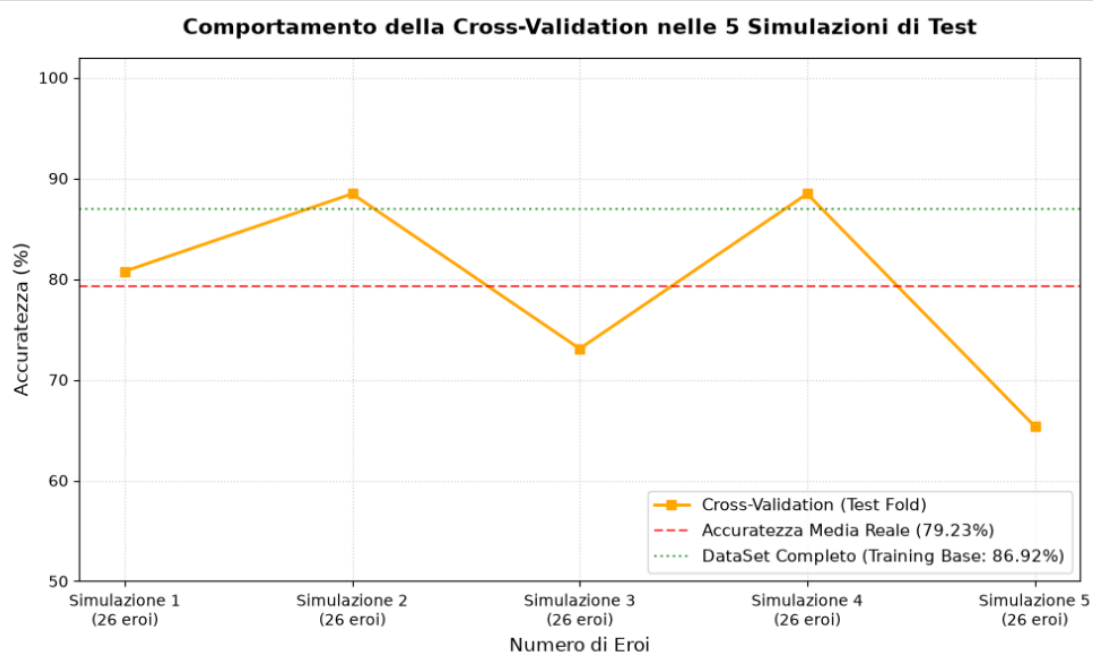
Questa evidenza convalida la scelta progettuale dell'iperparametro pari a 3: il modello apprende le regole generali del dominio senza specializzarsi eccessivamente.

Per la valutazione comparativa del modello è stata privilegiata l'Accuratezza Media (*Accuracy*) accompagnata dalla Deviazione Standard sui 5 fold. Essendo il dataset bilanciato tra le classi di ruolo, l'accuratezza offre un indicatore globale sintetico ed efficace.

Una prima valutazione sul set di addestramento ha mostrato un'accuratezza del 86% circa, dato però non significativo poichè calcolato su dati già visti dal modello. La valutazione affidabile è quella ottenuta dalla Cross-validation.

Verifichiamo se con la Cross-Validation questi parametri prestazionali mantengono stabilità. Sottoponendo l'albero alle varie simulazioni di test indipendenti previsti dal protocollo di validazione.

E' possibile visionare il grafico della Cross-Validation sulle 5 simulazioni:

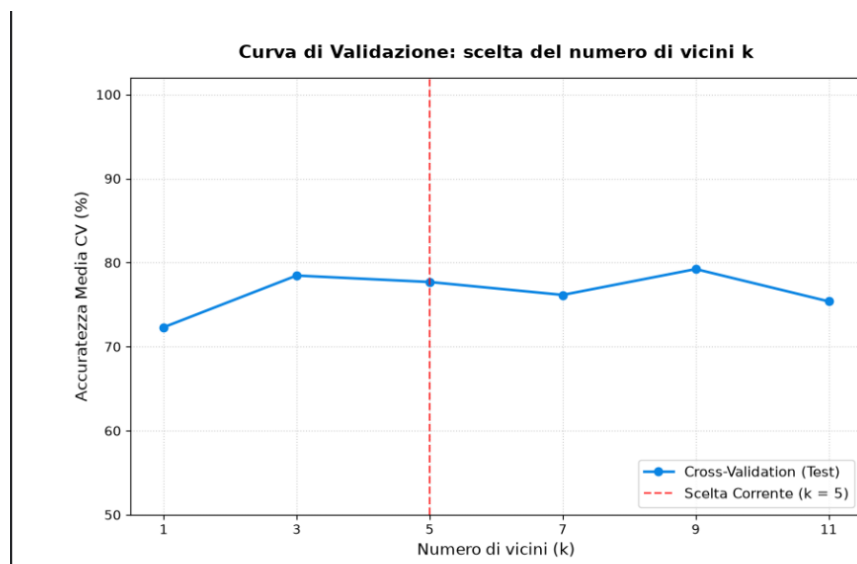


L'aspetto più rilevante dell'esperimento risiede nella *Deviazione Standard*, pari a $\pm 8.97\%^{**}$. Questa marcata oscillazione delle performance tra le singole simulazioni, che spaziano dai picchi d'eccellenza stabili delle Simulazioni 2 e 4 (88.46%) fino al crollo critico registrato nella Simulazione 5 (65.38%), evidenzia una **forte sensibilità** dell'albero alla combinazione dei dati estratti.

RISULTATI CROSS-VALIDATION	
Simulazione 1 (su 26 eroi):	80.77%
Simulazione 2 (su 26 eroi):	88.46%
Simulazione 3 (su 26 eroi):	73.08%
Simulazione 4 (su 26 eroi):	88.46%
Simulazione 5 (su 26 eroi):	65.38%
Accuratezza MEDIA Reale:	79.23%
Deviazione Standard :	+/- 8.97%

Nella prossima fase, grazie all'ausilio della componente semantica e del ragionatore deduttivo, controlleremo come sia possibile intervenire a posteriori su queste forti fluttuazioni prestazionali.

Valutazione del K-NN



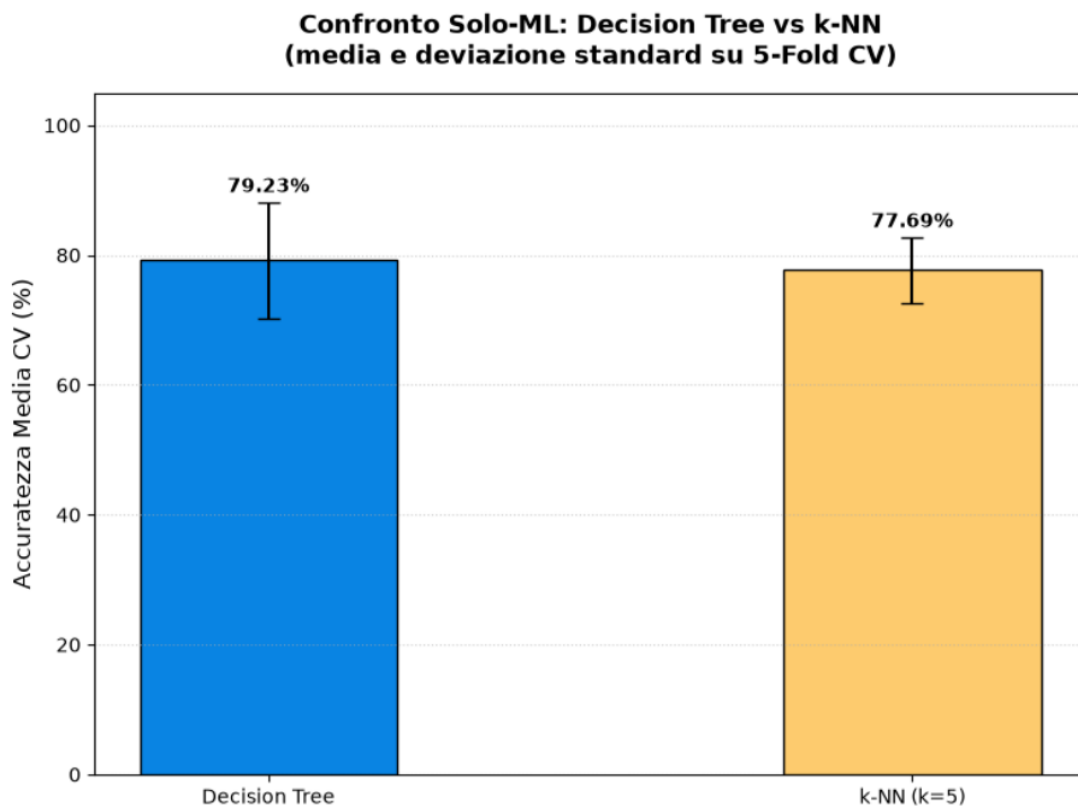
Dall'osservazione della curva di validazione relativa al classificatore *k-Nearest Neighbors* al variare del numero di vicini $k \in \{1, 3, 5, 7, 9, 11\}$, emergono importanti considerazioni teoriche e pratiche.

Per $k = 1$, l'accuratezza media risulta sensibilmente più bassa ($\approx 72.3\%$), evidenziando un chiaro fenomeno di *overfitting* dovuto all'elevata sensibilità dell'algoritmo verso il rumore e gli *outlier* del dataset. A partire da $k \geq 3$, la curva tende a stabilizzarsi all'interno di un intervallo di accuratezza ristretto tra il 75% e il 79%. Si è scelto di testare solo valori dispari di k per convenzione, riducendo — pur non azzerandola del tutto, essendo il problema a tre classi anziché binario — la probabilità di situazioni di parità nella votazione a maggioranza tra i ruoli.

Sebbene i valori $k = 3$ e $k = 9$ registrino picchi di accuratezza marginalmente superiori, la selezione di $k = 5$ ($\approx 77.7\%$) rappresenta il miglior compromesso tra stabilità e capacità di generalizzazione.

In termini generali, il k-NN dimostra una stabilità complessiva, con fluttuazioni contenute entro 3-4 punti percentuali nell'intervallo $3 \leq k \leq 11$. Tuttavia, questa stabilità apparente non basta a preferirlo come modello primario: la sua accuratezza media ($\approx 77.69\%$) rimane inferiore a quella ottenuta dal Decision Tree (79.23%) e la sua natura di classificatore metrico spaziale non consente di estrarre regole simboliche trasparenti da integrare con l'Ontologia OWL.

La scelta di $k = 5$ si attesta quindi come la configurazione ottimale per il k-NN, confermando al contempo la centralità dell'Albero di Decisione nell'architettura globale del sistema. Come mostrato in figura:



6 Ontologia

6.1 Sommario

L'esigenza di strutturare il sapere relativo all'universo dei supereroi ha spinto verso l'adozione di un approccio ontologico, ideale per mappare in modo formale la gerarchia delle classi, i legami semantici e le singole entità degli universi Marvel e DC.

Scritta in linguaggio OWL, l'infrastruttura si divide nettamente tra la *TBox* (definisce vincoli e regole dei profili tattici), e la *ABox* (accoglie i dati e le caratteristiche specifiche di ogni personaggio).

Questa base di conoscenza coopera con l'algoritmo predittivo, rettificandone le anomalie statistiche.

6.2 Strumenti Utilizzati

Questa parte di progetto utilizza un editor per la creazione di ontologia: [Protégé](#). Dopo la creazione viene esportata l'ontologia in formato OWL.

Ci serviamo della libreria: `owlready2`. Essa permette di caricare ontologie OWL 2.0 come oggetti Python, modificarle, salvarle ed eseguire ragionamenti tramite architettura HermiT.

6.3 Decisioni di Progetto

La specificità del dominio d'indagine ha richiesto la modellazione da 0 dell'ontologia. La classe fondamentale è rappresentata da **Character**, da cui si diramano le categorie tattiche e le caratteristiche descrittive.

Classi Definite e Modellazione dei Ruoli Tattici

Nel modello ontologico sviluppato, i ruoli principali non sono semplici classi primitive asserite, ma *classi definite* mediante relazioni di equivalenza ($\equiv$, `EquivalentClasses` in OWL), che abilitano la riclassificazione automatica da parte di HermiT:

- **Leader** ($\equiv \text{Character} \sqcap \text{Charismatic} \sqcap \text{StrategicGenius}$): Un personaggio viene inferito come **Leader** se possiede un'intelligenza strategica elevata (`hasIntelligence` ≥ 8) unitamente a una spiccata popolarità o carisma (`hasPopularity` ≥ 8).
- **Powerhouse** ($\equiv \text{Character} \sqcap \text{DevastatingForce} \sqcap \neg \text{StrategicGenius}$): Definito come un eroe caratterizzato da una forza fisica devastante (`hasStrength` ≥ 7) che tuttavia non ricade nella categoria dei geni strategici ($\neg \text{StrategicGenius}$).
- **Specialist**: A differenza delle prime due, la classe **Specialist** è lasciata volutamente come classe primitiva (`SubClassOf Character`) priva di vincoli numerici rigidi. I personaggi che non presentano pattern parametrici dominanti vengono gestiti dal sistema tramite un meccanismo logico di *fallback*.

Per garantire il rigore semantico, l'ontologia impone assiomi di disgiunzione formale (`DisjointClasses`):

$$\text{Disjoint}(\text{Leader}, \text{Powerhouse}, \text{Specialist}) \quad (1)$$

Tale vincolo impedisce a un individuo di appartenere simultaneamente a più ruoli tattici. Sebbene le soglie numeriche (`xsd:integer`) per l'intelligenza o la forza coincidano con alcune variabili usate dall'apprendimento statistico, l'ontologia non costituisce una duplicazione del target: il suo

valore risiede nell'imporre un vincolo di veridicità assiomatica e deterministica che sovrascrive gli errori di campionamento o di generalizzazione del Decision Tree.

Inferenza Avanzata: Property Chain e Vulnerabilità

Un elemento qualificante dell'ontologia è la deduzione delle vulnerabilità tattiche senza alcuna asserzione diretta nel dataset. Sfruttando il costruttore di catene di proprietà (`ObjectPropertyChain`), è stata definita la seguente regola:

$$\text{hasPowerSource} \circ \text{hasWeakness} \sqsubseteq \text{hasVulnerability} \quad (2)$$

Unitamente alle definizioni di background-knowledge asserite sulle fonti di potere (`TechnologicalWeapon` con debolezza `EMP`, e `Supernatural` con debolezza `AntiMagicSeal`), il reasoner `HermiT` ricava automaticamente:

- **EMPTarget:** eroi la cui fonte energetica è tecnologica, dedotti come vulnerabili ad attacchi ad impulso elettromagnetico.
- **AntiMagicSealTarget:** eroi di origine soprannaturale, dedotti come sensibili ai sigilli anti-magici.

Integrazione e Automazione nel Flusso Python

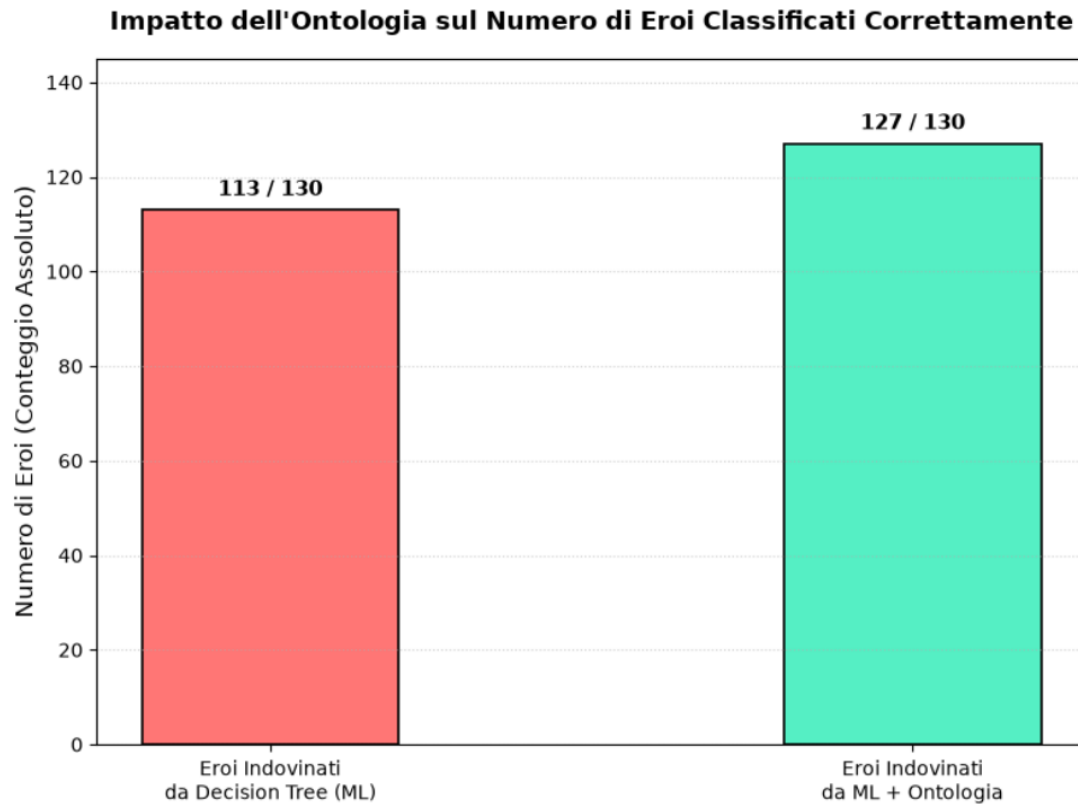
L'integrazione programmatico-semantic è gestita all'interno del codice Python dai due metodi principali:

- `populate_ontology()`: azzerà il workspace logico, carica la TBox vuota (`super_heroes_empty.owl`) e la popola leggendo il dataset CSV. Crea le istanze individuali della classe `Character`, associa i dati numerici (`hasStrength`, `hasIntelligence`, `hasSpeed`, `hasPopularity`) e collega le *Object Properties* (`hasPowerSource`, `hasUniverse`). Il grafo risultante viene salvato nel file ABox `super_heroes_populated.owl`.
- `run_reasoning()`: carica l'ontologia popolata ed esegue il reasoner `HermiT` via `sync_reasoner_hermit()`. Interroga le classi inferite indirettamente (`INDIRECT_IS_A`) per estrarre il ruolo dedotto ed estrae gli insiemi `EMPTarget` e `AntiMagicSealTarget` generati dalla *property chain*, restituendo una struttura dati integrata al modulo principale.

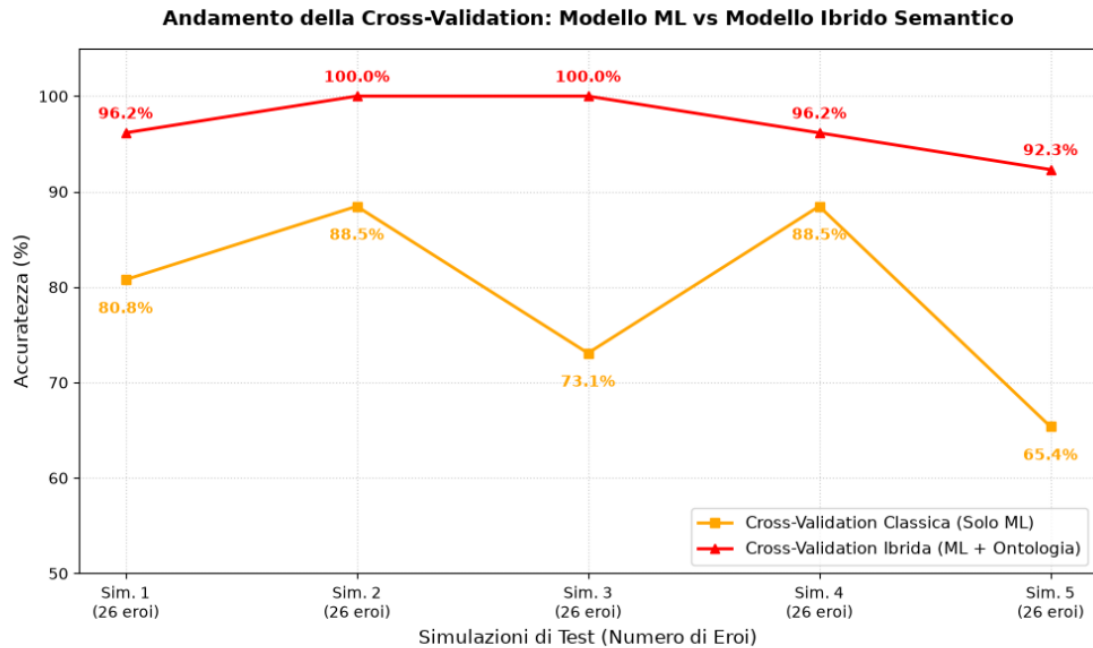
6.4 Valutazione

La base di conoscenza ontologica apporta un valore aggiunto all'interno dell'ecosistema ibrido. Il suo impatto effettivo viene dettagliato e dimostrato nella sezione conclusiva del lavoro, in cui si evidenzia come l'integrazione del ruolo tattico inferito dal ragionatore determini un drastico incremento nelle performance predittive e nella stabilità complessiva dei modelli di Machine Learning.

Infatti possiamo visualizzare dei grafici che attestino tale incremento:



Anche se a prima vista l'aumento degli eroi indovinati può sembrare ridotto (si passa da 113 a 127 eroi indovinati, su un totale di 130), la scomposizione della metrica sui singoli *fold* rivela un miglioramento strutturale legato alla **robustezza** e alla **stabilità** del sistema.



L'andamento del Modello Ibrido Semantico (rappresentato dalla curva rossa) dimostra l'efficacia dell'azione coordinata tra apprendimento statistico e vincoli logici:

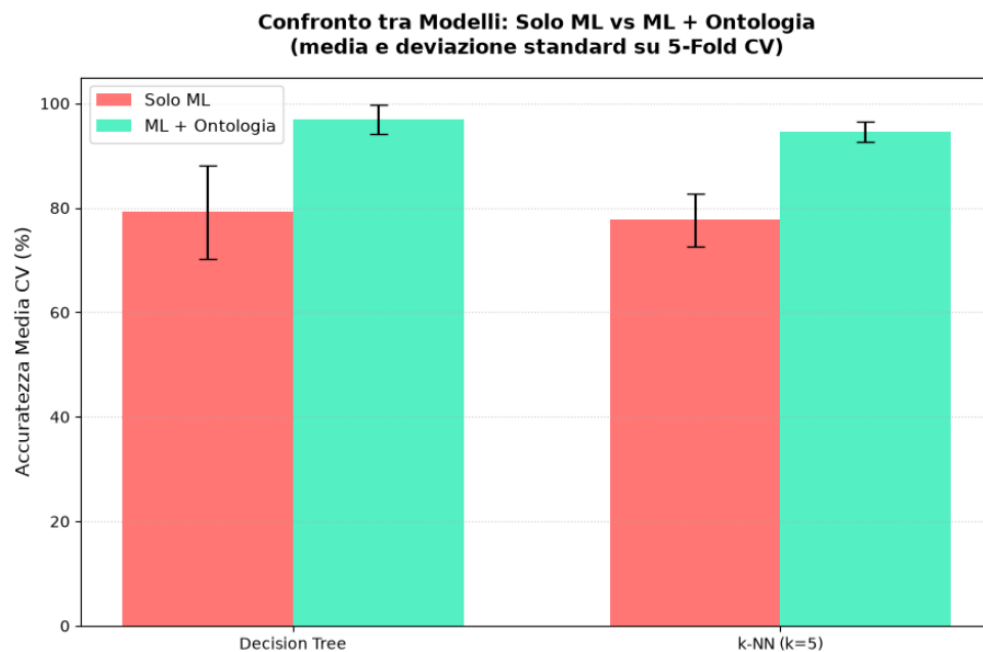
Il contributo dell'ontologia non consiste nella duplicazione delle informazioni già presenti nel dataset, ma agisce come meccanismo di validazione esterno e deterministico. Esso è capace di intervenire selettivamente sui casi in cui il modello statistico fallisce, come dimostrato.

Inoltre, l'obiettivo risiede nella capacità di **stabilizzare il sistema contro le anomalie dei dati**, eliminando i picchi negativi di errore.

Confronto modelli: Decision Tree e K-NN

Un'ulteriore conferma della validità della scelta del Decision Tree rispetto al k-NN, l'analisi comparativa delle metriche aggregate sui 5 fold (la figura sottostante) evidenzia due aspetti fondamentali:

- **Prestazioni Assolute del Modello Ibrido:** Sebbene entrambi i modelli traggano un giovamento sostanziale dall'integrazione semantica, il Decision Tree accoppiato all'Ontologia raggiunge il livello più alto di accuratezza media ($\approx 97\%$), superando la controparte k-NN ($\approx 94.5\%$).
- **Effetto Stabilizzatore sulla Varianza:** Nel regime di puro Machine Learning, il Decision Tree mostra una deviazione standard elevata (rappresentata dall'ampia barra di errore in rosso), a testimonianza della sua sensibilità alle fluttuazioni dei dati nei singoli fold. L'innesto dei vincoli logici dell'ontologia non solo eleva la media predittiva, ma riduce drasticamente l'incertezza del modello (barra nera del modello ibrido in verde), abbattendo la varianza e rendendo l'albero di decisione il classificatore primario ideale per l'architettura complessiva.



7 Sistema di Supporto alle Decisioni

7.1 Sommario

L'ultima fase del progetto concretizza il trasferimento tecnologico della conoscenza modellata, trasformando le inferenze della Description Logic (DL) e i dati del dataset in uno strumento di supporto alle decisioni (**Decision Support System - DSS**) interattivo. Il sistema consente all'utente di simulare scenari tattici e formulare *Query Semantiche* complesse tramite un'interfaccia a riga di comando.

Gli scenari tattici supportati comprendono:

1. **Missione Segreta (Infiltrazione spionistica):** estrazione diretta delle istanze appartenenti alla classe composta OWL *IdealInfiltrator*;
2. **Simulatore di Battaglia (Marvel vs DC):** reclutamento di coalizioni tattiche con vincoli di universo e copertura parziale dei ruoli;
3. **Leader Ideale (Attacco Cyber):** identificazione dei comandanti tecnologici tramite la classe composta *IdealCyberCommander*;
4. **Contromisura EMP (Bersagli vulnerabili):** individuazione delle minacce tecnologiche esposte a impulsi elettromagnetici;
5. **Contromisura Anti-Magica (Bersagli vulnerabili):** individuazione degli eroi di origine soprannaturale sensibili a sigilli mistici.

L'architettura dimostra come l'ontologia non sia un semplice repository statico, ma un motore logico capace di guidare sia il calcolo esatto sia la selezione euristica nelle decisioni strategiche della *Battle*.

7.2 Strumenti Utilizzati

Per la realizzazione dell'infrastruttura del Decision Support System si fa affidamento sull'ecosistema Python e sulle seguenti componenti software:

- **owlready2 e Reasoner HerMiT:** costituiscono il motore di inferenza semantica che precede l'esecuzione del DSS, responsabile di risolvere preliminarmente gli assiomi delle classi composte (*IdealInfiltrator*, *IdealCyberCommander*) e le catene di proprietà (*EMPTarget*, *AntiMagicSealTarget*);
- **Dizionari e Mappatura In-Memory (heroes_map):** struttura dati fondamentale utilizzata dal modulo per indicizzare in tempo reale i profili dei personaggi, memorizzando sia le flag booleane derivate dalle deduzioni formali della TBox, sia i criteri elementari necessari alla fase di valutazione euristica;
- **Funzioni Native di Ordinamento e Aggregazione (Python Standard Library):** impiegate nella FASE 2 del sistema per calcolare dinamicamente il punteggio di copertura dei criteri (`copertura = soddisfatti / len(criteri_fallback)`) e ordinare le candidature tramite ordinamento multivalore (`sort(key=lambda t: (t[0], t[1]))`), garantendo l'estrazione efficiente dei candidati ottimali per prossimità tattica.

7.3 Decisioni di Progetto

La componente strategica dell'architettura è racchiusa nel modulo `decision_system()`, concepito secondo un modello di ragionamento a due livelli che combina il rigore assiomatico della descrizione logica con la flessibilità dell'algoritmica euristica.

Architettura di Ragionamento a Due Livelli

Per superare i limiti dei sistemi tradizionali il DSS applica la seguente gerarchia decisionale:

1. **Livello 1 - Deduzione Logica Esatta (TBox DL Reasoning):** Il sistema interroga in prima battuta i verdetti calcolati dal ragionatore HermiT. Se la query richiede un profilo complesso definito nella TBox mediante assiomi formali ($\equiv$), il ragionatore verifica la membership formale dell'individuo. Se esistono elementi che soddisfano esattamente l'assioma, il sistema restituisce un match logico con confidenza al 100%.
2. **Livello 2 - Ranking per Prossimità Euristica (Fallback Mode):** Qualora nessun individuo soddisfi la totalità dei vincoli formali espressi nella TBox (o quando si operi sotto vincoli esterni come l'appartenenza a uno specifico universo), il sistema attiva un meccanismo di fallback. Tale livello calcola un indice di copertura euristica ($C \in [0, 1]$) misurando la percentuale di criteri elementari soddisfatti dall'eroe rispetto ai requisiti dello scenario:

$$C = \frac{\sum_{i=1}^N \mathbb{I}(\text{criterio}_i)}{N} \quad (3)$$

dove N è il numero di tratti richiesti e $\mathbb{I}$ è la funzione indicatore di possesso del tratto.

Integrazione delle Classi Composte e Property Chain dell'Ontologia

L'ontologia partecipa attivamente alla formulazione della risposta attraverso due meccanismi semantici fondamentali modellati nella TBox:

- **Classi Composte Equivalenti (EquivalentClasses):** Negli Scenari 1 e 3, il sistema non ricerca semplici combinazioni di attributi numerici, ma interroga classi strutturate create dal Reasoner:

`IdealInfiltrator  $\equiv$  Specialist  $\sqcap$  LowProfile  $\sqcap$  HighMobility`

`IdealCyberCommander  $\equiv$  Leader  $\sqcap$   $\exists$ hasPowerSource.TechnologicalWeapon`

- **Inferenza di Vulnerabilità via Catena di Proprietà (ObjectPropertyChain):** Negli Scenari 4 e 5, le vulnerabilità non sono scritte nel dataset grezzo. Il DSS individua i bersagli `EMPTarget` e `AntiMagicSealTarget` sfruttando la composizione logica inferita da HermiT:

`hasPowerSource  $\circ$  hasWeakness  $\sqsubseteq$  hasVulnerability`

I personaggi dotati di arma tecnologica ereditano la debolezza all'EMP, mentre quelli di origine soprannaturale ereditano la sensibilità al sigillo anti-magico, permettendo al DSS di suggerire contromisure tattiche matematicamente e logicamente indiscutibili.

7.4 Valutazione

La validazione del Decision Support System è stata condotta simulando le interazioni utente sul terminale e analizzando i verbali di risposta generati dal sistema. Successivamente viene illustrato il menu, il quale visualizza per ciascuna opzione sia il titolo sia la regola logica di fondo garantendo la totale trasparenza del processo decisionale.

```
Scegli lo scenario!
[1] MISSIONE SEGRETA (Infiltrazione spionistica)
    -> Estrazione diretta della classe composta OWL 'IdealInfiltrator' (Specialist n LowProfile n HighMobility).
[2] SIMULATORE DI BATTAGLIA: SQUADRA MARVEL vs MINACCIA DC
    -> Genera la coalizione ottimale Marvel valutando sia profilazione esatta sia copertura parziale dei ruoli.
[3] LEADER IDEALE (Attacco Cyber)
    -> Estrazione diretta della classe composta OWL 'IdealCyberCommander' (Leader n EhasPowerSource.TechnologicalWeapon).
[4] CONTROMISURA EMP (Bersagli vulnerabili)
    -> Identifica le istanze della classe dedotta EMPtarget via Property Chain (hasPowerSource - hasWeakness E hasVulnerability).
[5] CONTROMISURA ANTI-MAGICA (Bersagli vulnerabili)
    -> Identifica le istanze della classe dedotta AntiMagicSealTarget via Property Chain.
[0] Esci dal programma
```

Figura 2: Interfaccia da terminale della fase di Battle con la descrizione degli scenari e dei relativi assiomi DL.

```
Inserisci il numero dello scenario desiderato: 1

-----
RISPOSTA ALLA QUERY SEMANTICA: MISSIONE SEGRETA (Infiltrazione spionistica)
MODALITÀ ELABORAZIONE: FALLBACK EURISTICO (Nessun match esatto in TBox, ranking per prossimità)
-----
Membro Suggerito      | Classe OWL      | Universo  | Esito Reasoning / Match
-----
-> Superman            | Specialist      | DC        | PROSSIMITÀ EURISTICA (67%)
-> Spiderman           | Specialist      | Marvel    | PROSSIMITÀ EURISTICA (67%)
-> Flash               | Specialist      | DC        | PROSSIMITÀ EURISTICA (67%)
-> Green Arrow         | Specialist      | DC        | PROSSIMITÀ EURISTICA (67%)
-> Human Torch         | Specialist      | Marvel    | PROSSIMITÀ EURISTICA (67%)
-----
ESITO: Infiltrati identificati direttamente tramite la classe composta IdealInfiltrator.
```

Figura 3: Output del terminale per lo Scenario 1: identificazione degli infiltrati tramite la classe OWL *IdealInfiltrator*.

Prendendo come caso di studio l'esecuzione dello **Scenario 1**

- Il DSS interroga direttamente la classe OWL composta *IdealInfiltrator*, definita in TBox come *Specialist* $\sqcap$ *LowProfile* $\sqcap$ *HighMobility*;
- HermiT identifica con un match esatto gli eroi che appartengono simultaneamente alle tre classi che compongono la definizione, ossia gli eroi con basso profilo pubblico e alta mobilità classificati come *Specialist*;
- I candidati posizionati ai primi posti vengono riconosciuti come membri della classe OWL dedotta e ordinati alfabeticamente in modo deterministico, garantendo la riproducibilità dei risultati visualizzati a schermo.

In conclusione, l'integrazione del ragionatore nell'infrastruttura decisionale eleva il sistema da un semplice algoritmo di ordinamento a un vero e proprio assistente tattico semantico, in grado di spiegare la logica formale alla base di ogni raccomandazione fornita.

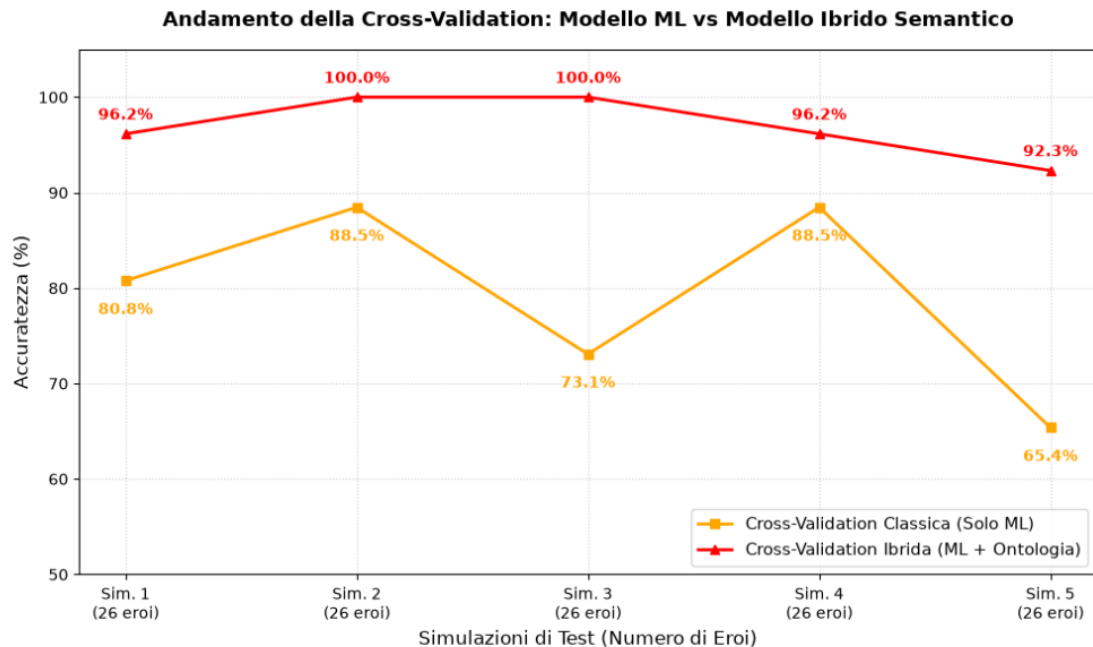
8 Conclusioni e Sviluppi futuri

'*HEROES IN SUPER-TRAINING*' ha dimostrato con successo l'efficacia e la robustezza dell'integrazione tra tecniche di Machine Learning statistico e metodologie di modellazione semantica basate su ontologie OWL, applicate alla strutturazione di un *Decision Support System* (DSS) nel dominio dei supereroi.

Il **Machine Learning**, pur mostrando un'elevata capacità di apprendimento dai dati storici grezzi, soffriva di una marcata instabilità nei casi di test più complessi, registrando crolli verticali nell'accuratezza (fino al 65.4%).

L'**Infrastruttura Semantica** basata sulla TBox OWL e sul ragionatore HermiT ha introdotto un livello di determinismo logico essenziale, agendo come un vincolo algoritmico a protezione delle anomalie statistiche.

I risultati della Cross-Validation a 5 fold hanno confermato la validità di questa architettura ibrida, come già osservato nel grafico seguente:



L'ontologia ha dunque dimostrato il suo valore non come classificatore isolato, ma come fondamentale supporto cooperativo per il modello di Machine Learning.

Lo sviluppo del modulo `decision_system()` ha permesso di tradurre la conoscenza estratta in uno strumento operativo concreto.

Sviluppi futuri?

Un promettente sviluppo futuro per il sistema ibrido consiste nell'integrazione di **Datalog** (o di linguaggi di regole basati su di esso, come SWRL - *Semantic Web Rule Language*).

L'introduzione di un motore di ragionamento basato su Datalog permetterebbe di modellare dinamiche di combattimento più avanzate, di formalizzare le euristiche spostandole direttamente all'interno dello strato logico formale della Base di Conoscenza.

In sintesi si eleverebbe la complessità e il realismo delle simulazioni all'interno della fase di *Battle*.

9 Bibliografia

Riferimenti bibliografici

- [1] Poole, D., & Mackworth, A. *Supervised Machine Learning*.
In: Artificial Intelligence: Foundation of Computational Agents, Capitolo 7.
- [2] Poole, D., & Mackworth, A. *Knowledge Graph and Ontologies*.
In: Artificial Intelligence: Foundation of Computational Agents, Capitolo 16.
- [3] Oracle Corporation. *MySQL Database Service*.
<https://www.mysql.com/it/>
- [4] Pandas. *Getting Started*.
https://pandas.pydata.org/docs/getting_started/index.html#getting-started
- [5] Pandas. *User Guide*.
https://pandas.pydata.org/docs/user_guide/index.html
- [6] Stack Overflow. *Convert text columns into numbers in sklearn*.
<https://stackoverflow.com/questions/34915813/convert-text-columns-into-numbers-in-sklearn>
- [7] Scikit-learn. *DecisionTreeClassifier*.
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- [8] Scikit-learn. *KNeighborsClassifier*.
<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
- [9] Scikit-learn. *LabelEncoder*.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>
- [10] Scikit-learn. *StandardScaler*.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- [11] Scikit-learn. *make_pipeline*.
https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.make_pipeline.html
- [12] Scikit-learn. *clone*.
<https://scikit-learn.org/stable/modules/generated/sklearn.base.clone.html>
- [13] Scikit-learn. *accuracy_score*.
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html
- [14] Scikit-learn. *cross_val_score*.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_val_score.html
- [15] Scikit-learn. *KFold*.
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.KFold.html

- [16] Python Software Foundation. *Mapping Types — dict*.
<https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
- [17] Python Software Foundation. *Sorting HOWTO — Sorting Techniques in Python*.
<https://docs.python.org/3/howto/sorting.html>
- [18] Matplotlib. *Quick Start*.
https://matplotlib.org/stable/users/explain/quick_start.html
- [19] Stanford Center for Biomedical Informatics Research. *Protégé Ontology Editor*.
<https://protege.stanford.edu/software/>
- [20] Lamy, J. B. (2017). *Owlready2: A Python module for ontology-oriented programming*.
Artificial Intelligence in Medicine, 80, 11-28.
<https://pypi.org/project/owlready2/>